INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA
SALT LAKE
(Affiliated to UEM Kolkata and Approved by AICTE)

Advanced Programming Lab
[PCCCS495]

Hospital Management System

Name: -Rhitam Biswas

Semester: -IV

Class Roll: - 46

Enrolment Number: - 12024052004044

Date: - 29/03/2026

Instructor name: - Susovan Jana, Soumadip Biswas, Swagatam Basu

Department of Information Technology

2026

2. Problem Statement

Modern hospitals require efficient systems to manage patients and provide appropriate treatment based on their condition. Manual handling can lead to delays, especially in emergency situations.

This project aims to develop a **console-based Hospital Management System** using Java that:

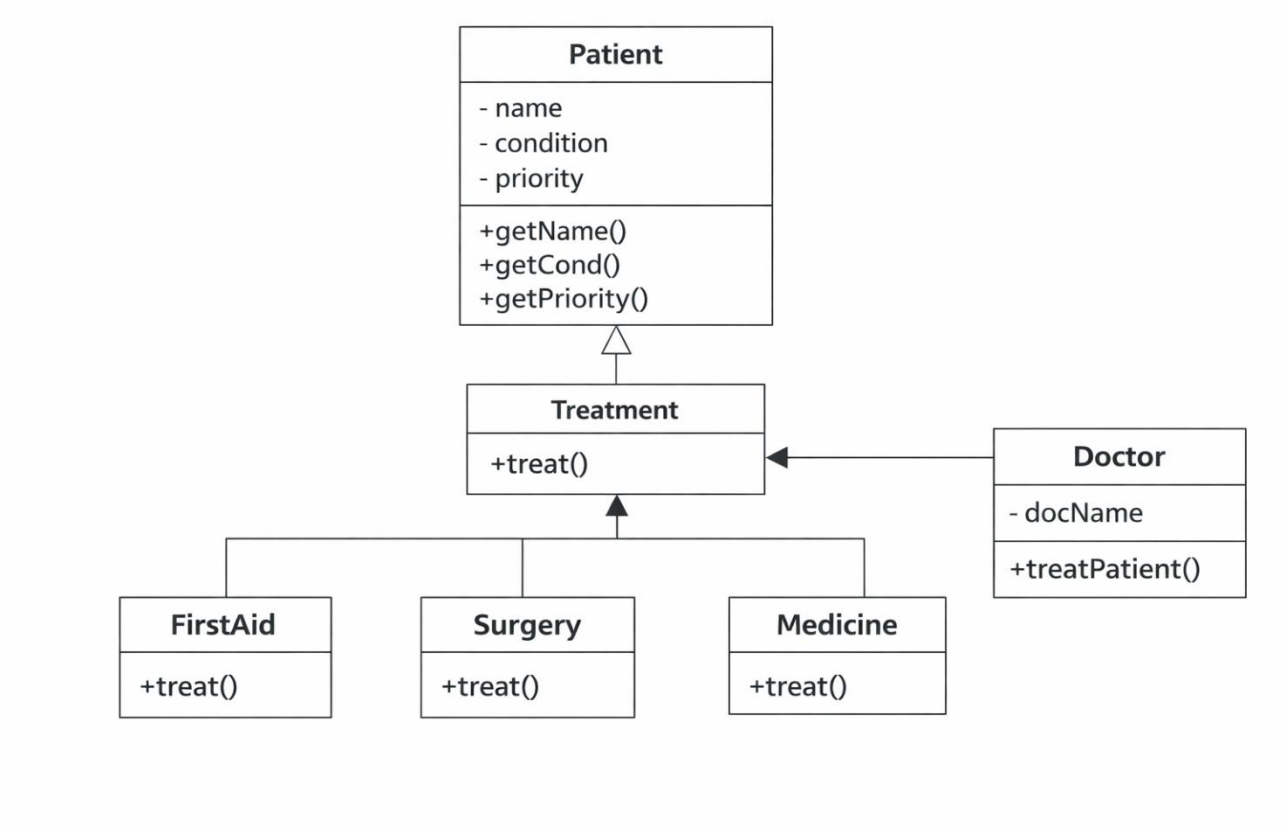
- Accepts patient details (name, disease, priority)
- Assigns a doctor
- Determines treatment based on condition and priority
- Handles critical cases with emergency surgery
- Ensures robust execution using exception handling

The goal is to demonstrate core **Object-Oriented Programming (OOP)** concepts in a real-world scenario.

---

3. System Design

3.1 UML Class Diagram (Text Representation)



3.2 Description of Major Classes

Patient

Stores patient details such as name, condition, and priority.

Doctor

Handles treatment decisions based on patient condition and priority.

Treatment (Base Class)

Defines a general treatment method.

FirstAid, Surgery, Medicine

Derived classes implementing specific treatments.

3.3 Package Structure

All classes are placed in a single package (default package) for simplicity:

Hospital Project

- Hospital.java
- Patient.java
- Doctor.java
- Treatment.java
- FirstAid.java
- Surgery.java
- Medicine.java

3.4 Interaction Flow

1. User enters patient details
2. Selects doctor
3. System creates Patient object
4. Doctor evaluates condition
5. Priority check:
    - Critical → Emergency Surgery
    - Normal → Medicine / First Aid
6. Treatment executed
7. Option to continue or exit

---

4. OOP Concepts Demonstrated

4.1 Encapsulation

Where used: Patient class

Why used: To protect data and allow controlled access

Code Snippet:

```
class Patient {
   private String name;
   private String cond;

   public String getName() {
      return name;
   }
}
```

Explanation:
Data members are private and accessed via getter methods, ensuring data security.

4.2 Inheritance

Where used: Treatment → FirstAid, Surgery, Medicine

Why used: To reuse common logic and extend functionality

Code Snippet:

```
class FirstAid extends Treatment {
   public void treat(Patient p) {
      System.out.println("Providing First Aid");
   }
}
```

Explanation:
Child classes inherit from Treatment and override behavior.

4.3 Polymorphism

Where used: Treatment reference

Why used: To call different methods using same reference

Code Snippet:

```
Treatment t;

t = new Surgery();

t.treat(p);
```

Explanation:
Same method behaves differently depending on object type.

4.4 Abstraction

Where used: Treatment class

Why used: To define a common interface for all treatments

Code Snippet:

```
class Treatment {
   public void treat(Patient p) {
      System.out.println("General treatment");
   }
}
```

Explanation:
Provides a base structure for specialized treatments.

4.5 Exception Handling

Where used: Main method

Why used: To prevent program crash

Code Snippet:

```
try {
   int choice = sc.nextInt();
} catch (Exception e) {
   System.out.println("Invalid input");
}
```

Explanation:
Handles invalid inputs safely.

---

5. Implementation Highlights

Snippet 1: Doctor Selection

```
switch(choice) {
   case 1: d = new Doctor("Dr.Arijit Deb"); break;
}
```

Justification: Ensures structured and readable selection logic.

Snippet 2: Priority Handling

```
if(p.getPriority().equalsIgnoreCase("critical")) {
   t = new Surgery();
}
```

Justification: Critical patients are prioritized for emergency treatment.

Snippet 3: Condition-Based Treatment

```
if(cond.equals("fever")) {
   t = new Medicine();
}
```

Justification: Enables dynamic treatment assignment.

Snippet 4: Loop Control

```
if(!again.equalsIgnoreCase("yes")) {
   break;
}
```

Justification: Allows continuous execution until user exits.

---

6. Testing & Error Handling

Test Cases

Input	Expected Output
Fever + Normal	Medicine
Injury + Normal	First Aid
Heart + Critical	Surgery

Edge Cases

- Invalid doctor selection
- Empty input
- Wrong priority value

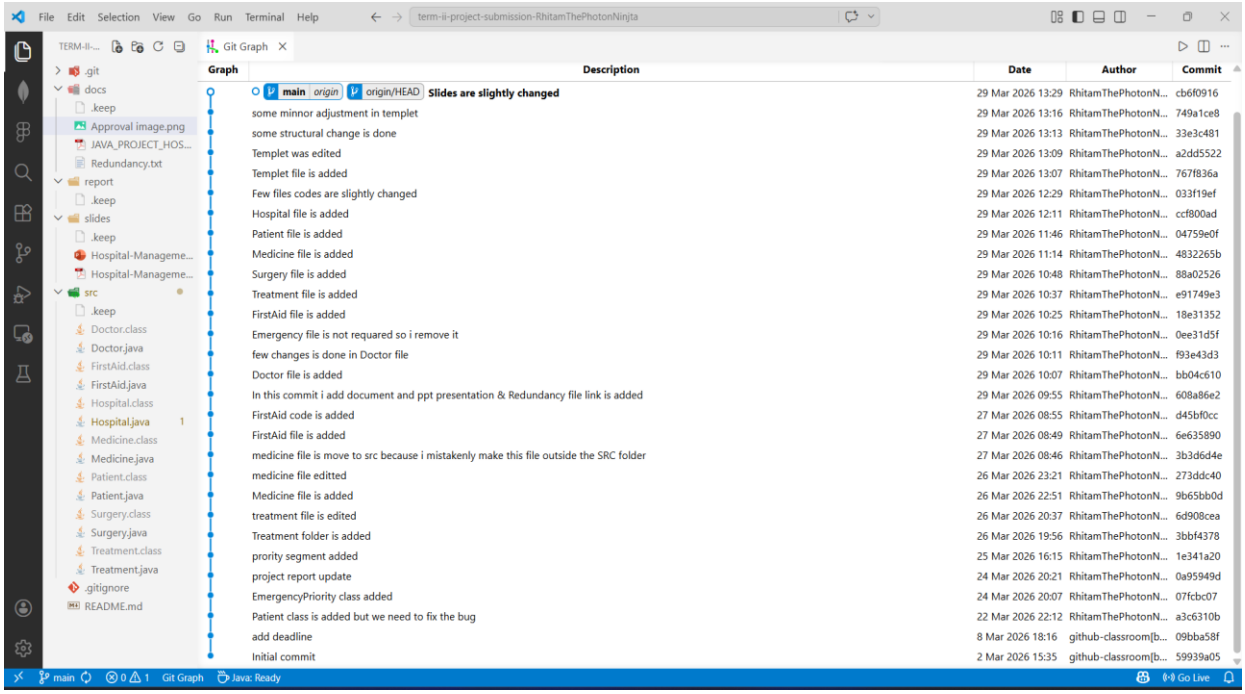
Failure Scenarios

- Non-integer doctor input → handled by try-catch
- Unexpected condition → default treatment

---

7. Git Workflow

Commit History



Development Progression

Project was developed incrementally:

1. Basic structure
2. OOP implementation
3. Error handling
4. Feature enhancement

---

8. Conclusion & Future Scope

Conclusion

The project successfully demonstrates OOP principles through a real-world hospital system. It efficiently handles patient data, prioritization, and treatment assignment.

Future Scope

- GUI using Java Swing
- Database integration
- Multi-doctor specialization
- Online appointment system
- AI-based diagnosis

9.Appendix

Initial Project Proposal

JAVA_PROJECT_HOSPITAL (INITIAL PROPOSAL)

Review Mail

approval email with APPROVED_MINOR

APPROVAL_IMAGE

Additional Submissions (if any)

N.A
